



Skill: dsh

➤ "Referencia del Agent Harness DeepSeek (dsh) — kernel de plugins Cordis, 4 modos runtime, trazabilidad append-only, instalación y operación. Úsalo cuando trabajes con el harness dsh, perfiles, bundles, plugins Cordis, ev

dsh — Agent Harness: Cordis Plugin Kernel

Cuándo usar este skill

Usa este skill cuando necesites:

- Entender la arquitectura del harness dsh (kernel Cordis, seams, servicios core).
- Trabajar con perfiles (`dsh -profile`), bundles, parches y overlays.
- Gestionar plugins (`dsh plugin`), subagentes (Codex, Claude Code, ACP, SDK).
- Implementar servicios remotos con Typert (`@Remote`, `@RemoteScope`).
- Consumir eventos del ciclo de vida del agente (`agent/`, `session/`, `tools/`).
- Configurar plugins en `cordis.yml` con sus interfaces `Config`.
- Operar los 4 modos runtime: web, headless, sdk, acp.

Estructura del proyecto

El harness se organiza en paquetes npm bajo el scope `@deepseek-ai/`:

- **Core:** `dsh-agent`, `dsh-agent-loop`, `dsh-session`, `dsh-tools`, `dsh-system-prompt` — servicios dueños del estado.
- **Seams:** `dsh-llm`, `dsh-fs`, `dsh-storage`, `dsh-subagents`, `dsh-compaction`, `dsh-sandbox` — interfaces abstractas con múltiples backends.
- **Bundles:** `dsh-base`, `dsh-web-app`, `dsh-headless`, `dsh-sdk-app`, `dsh-sdk-minimal`, `dsh-acp-app` — composiciones de aplicación.
- **Protocolo:** `dsh-typert-protocol` — RPC tipado entre Host y Cliente.
- **Frontend:** `dsh-client-*`, `dsh-host-webserver`, `dsh-web-app`.

API principal

Contexto Cordis (ctx)

```
1 // Ciclo de vida
2 ctx.extend(meta?: {}): this
3 ctx.isolate(name: string, label?: symbol): this
4 ctx.intercept(name: string, config: any): this
5
6 // Servicios
7 ctx.get(name: string, strict?: boolean): any
8 ctx.set(name: string, value: any): void
9 ctx.provide(name: string, value?: any): () => void
10 ctx.accessor(name: string, options: { get?, set? }): void
11 ctx.mixin(name: string, mixins: string[] | Dict<string>): void
12
13 // Plugins
14 ctx.plugin(plugin: Plugin, ...args: any[]): Fiber & PromiseLike<Fiber>
15 ctx.inject(deps: Inject, callback: Plugin.Function<void>): Fiber &
    PromiseLike<Fiber>
16
17 // Efectos
18 ctx.effect(execute: () => SyncEffect, label?: string): Disposable<
    Promise<void>>
19 ctx.effect(execute: () => Effect, label?: string): AsyncDisposable<
    Promise<void>>
```

Eventos

```
1 ctx.parallel(name: string, ...args: any[]): Promise<void> //
    concurrente
2 ctx.emit(name: string, ...args: any[]): void // síncrono
3 ctx.serial(name: string, ...args: any[]): Promise<any> //
    secuencial async
4 ctx.bail(name: string, ...args: any[]): any //
    secuencial sync
5 ctx.waterfall(name: string, ...args: any[]): any //
    middleware encadenado
6 ctx.on(name: string, listener: Function, options?: boolean |
    EventOptions): () => boolean
7 ctx.once(name: string, listener: Function, options?: boolean |
    EventOptions): () => boolean
```

Typert Remote

```

1  class TypertRemoteService {
2      constructor(ctx: Context, namespace: string)
3  }
4
5  function bindTypertRemote(service: any, serviceKey: string): void
6
7  // Decoradores
8  @Remote(methodName: string)           // endpoint RPC unario
9  @RemoteScope(key: string, methodName: string) // RPC con ámbito (ej. '
    agent')
10
11 // Cliente
12 ctx.remote.<namespace>.<method>(...)
13 agentCtx.remote.<namespace>.<method>(...)

```

Service (clase base)

```

1  class Service {
2      public name!: string
3      constructor(ctx: Context, name: string)
4      static readonly init: unique symbol
5      static readonly check: unique symbol
6      static readonly config: unique symbol
7      static readonly invoke: unique symbol
8      static readonly extend: unique symbol
9      static readonly tracker: unique symbol
10     static readonly resolveConfig: unique symbol
11 }

```

Fiber

```

1  fiber.uid: number | null
2  fiber.ctx: Context
3  fiber.config: any
4  fiber.state: FiberState // PENDING      LOADING      ACTIVE
                        UNLOADING      DISPOSED
5  fiber.dispose(): Promise<void>
6  fiber.name: string
7  fiber.store: Dict<Impl> | undefined
8  fiber.inertia: Promise<void> | undefined
9  fiber.assertActive(): void
10 fiber.getEffects(): EffectMeta[]
11 fiber.await(): Promise<Fiber>
12 fiber.restart(): Promise<void>
13 fiber.update(config: any, noSave?: boolean): any

```

Herramientas

```
1 ctx.tools.register(toolDefinition): Disposer
2 ctx.tools.restrict(scopedRegistration): void
3 ctx.tools.guard(exec => boolean): void
4 ctx.tools.execute(params: {
5   callId: ToolCallId
6   name: string
7   arguments: Record<string, unknown>
8   signal: AbortSignal
9 }): Promise<{ content: ContentBlock[] }>
```

Patrones de uso

1. Registrar un servicio

```
1 declare module '@deepseek-ai/cordis' {
2   interface Context { greeter: GreeterService }
3 }
4
5 export class GreeterService extends Service {
6   constructor(ctx: Context) { super(ctx, 'greeter') }
7   greet(who: string) { return 'Hello, ${who}!' }
8 }
9
10 export function apply(ctx: Context) { ctx.plugin(GreeterService) }
```

2. Consumir con dependencia dura

```
1 export const inject = ['greeter']
2 export function apply(ctx: Context) {
3   console.log(ctx.greeter.greet('world'))
4 }
```

3. Waterfall para decisiones

```
1 ctx.on('some-decision-event', function (payload, next) {
2   if (payload.blocked) return { denied: true } // cortocircuito
3   return next()
4 })
```

4. Servicio remoto Typert

```

1 export class GoalService extends TypertRemoteService {
2   constructor(ctx: Context) { super(ctx, 'goals') }
3
4   @Remote('create')
5   createForClient(agent: Agent, request: { objective: string }, signal:
6     AbortSignal) {
7     signal.throwIfAborted()
7     return this.create(agent, request)
8   }
9
10  @RemoteScope('agent', 'current')
11  currentForClient() { return { accepted: true } }
12 }

```

5. Consumo remoto con manejo de errores

```

1 export const inject = ['remote', 'remote.notes']
2
3 const result = await ctx.remote.notes.list()
4 if (!result.ok) {
5   if (result.error.code === 'note/not-found') return []
6   throw result.error
7 }
8 return result.value

```

Configuración

Perfiles

```

1 dsh --profile web           # base + web-app, parches en vivo
2 dsh --profile headless     # base + headless, solo startup
3 dsh --profile sdk          # base + sdk-app, solo startup
4 dsh --profile sdk-minimal  # bundle independiente
5 dsh --profile acp          # base + acp-app, solo startup

```

Capas de configuración (orden de aplicación)

1. Parches de bundles en `dsh.profile.bundles`
2. `cordis.patch.yml` del perfil
3. `DSH_HOME/cordis.patch.ymlOverlays-patch <path>enorden`

Gestión de plugins

```
1 dsh plugin --profile <name> add <package>
2 dsh plugin --profile <name> remove <package>
3 dsh plugin --profile <name> add github:deepseek-harness/turtle-ui
```

cordis.yml

```
1 plugins:
2   '@deepseek-ai/dsh-agent-loop':
3     config:
4       maxParallelToolCalls: 4
5       agents:
6         - id: main-agent
7           provider: deepseek
8           model: deepseek-chat
9           cwd: /home/user/project
```

Ejemplos de código reutilizable

Ver [references/ejemplos.md](#) para fragmentos completos.

Glosario

Ver [references/glosario.md](#) para términos técnicos.

Prerrequisitos

Ver [references/prerrequisitos.md](#) para dependencias e instalación.

api

API principal del proyecto dsh

Contexto Cordis (ctx)

Objeto central: servicios, eventos y ciclo de vida accesibles a través de ctx.

Métodos de ciclo de vida

```
1 // Crear hijo con metadatos adicionales (herencia prototípica)
2 ctx.extend(meta?: {}): this
3
4 // Crear hijo con ámbito de servicio independiente para 'name'
5 ctx.isolate(name: string, label?: symbol): this
6
7 // Añadir configuración de interceptación para plugins bajo este
  contexto
8 ctx.intercept(name: string, config: any): this
```

Propiedades

```
1 ctx.root          // Contexto raíz de la aplicación (@experimental)
2 ctx.baseUrl       // URL base para resolver especificadores relativos
3 ctx.events        // Servicio de bus de eventos
4 ctx.logger        // Servicio de logging; usar ctx.logger(name) para
  logger nombrado
5 ctx.reflect       // Capa de reflexión (ctx.get, ctx.provide, ...)
6 ctx.registry      // Registro de plugins (ctx.plugin, ctx.inject)
```

Miembros estáticos

```
1 Context.effect    // Symbol para árbol de diagnóstico EffectMeta en
  disponer
2 Context.filter    // Symbol para filtro de escucha del contexto
3 Context.isolate   // Symbol del mapa de aislamiento
4 Context.intercept // Symbol del mapa de interceptación
5 Context.is(value) // Type guard: true si value es un contexto Cordis
```

Almacén de servicios y mixins

```
1 ctx.get(name: string, strict?: boolean): any
2 ctx.set(name: string, value: any): void
3 ctx.provide(name: string, value?: any): () => void
4 ctx.accessor(name: string, options: { get?, set? }): void
5 ctx.mixin(name: string, mixins: string[] | Dict<string>): void
```

API de Eventos

Métodos de despacho

```

1 ctx.parallel(name: string, ...args: any[]): Promise<void> //
   concurrente
2 ctx.emit(name: string, ...args: any[]): void // síncrono
   , ignora retorno
3 ctx.serial(name: string, ...args: any[]): Promise<any> //
   secuencial async hasta bail
4 ctx.bail(name: string, ...args: any[]): any //
   secuencial sync hasta bail
5 ctx.waterfall(name: string, ...args: any[]): any //
   middleware encadenado

```

Registro de listeners

```

1 ctx.on(name: string, listener: Function, options?: boolean |
   EventOptions): () => boolean
2 ctx.once(name: string, listener: Function, options?: boolean |
   EventOptions): () => boolean

```

Tipos

```

1 interface EventOptions {
2   prepend?: boolean // Añadir antes de listeners existentes
3   global?: boolean // Recibir evento ignorando filtros de contexto
4 }
5
6 type DispatchMode = 'emit' | 'parallel' | 'serial' | 'bail' | '
   waterfall'

```

Typert Remote Service

TypertRemoteService

Clase base para servicios de negocio expuestos al cliente. El constructor enlaza explícitamente la clave de servicio Cordis y el namespace Remoto por defecto.

```

1 constructor(ctx: Context, namespace: string)

```

bindTypertRemote(service, serviceKey)

Alternativa para servicios con otra clase base. Declara un binding público inspeccionable.

Decoradores

```
1 @Remote(methodName: string)
2 // Decorador para exponer un método de instancia público y no estático
  como endpoint RPC unario.
3
4 @RemoteScope(key: string, methodName: string)
5 // Decorador para métodos que operan sobre un contexto resuelto por á
  mbito (ej. 'agent'),
6 // sin recibir el objeto explícitamente.
```

Interfaces de cliente

```
1 ctx.remote.<namespace>.<method>(...) // llamadas RPC directas
2 agentCtx.remote.<namespace>.<method>(...) // llamadas RPC con ámbito
  de agente
```

Manejo de errores remotos

```
1 // Verificar result.ok; si es false, ramificar por result.error.code (
  no instanceof)
2 // Lanzar result.error para propagar errores remotos
3 // Usar isRemoteFailure(error) para distinguir fallos remotos de
  defectos locales
4 // Un aborto resulta en error con código gateway/cancelled
```

Service (class base)

```
1 class Service {
2   public name!: string
3   constructor(ctx: Context, name: string) // Registra la instancia en
    ctx.<name>
4
5   static readonly init: unique symbol // Método post-
    constructor (class plugins)
6   static readonly check: unique symbol // Predicado de
    disponibilidad para ctx.provide()
7   static readonly config: unique symbol // Phantom type para
    intercept-config
8   static readonly invoke: unique symbol // Cuerpo de llamada (
    ej. ctx.logger())
9   static readonly extend: unique symbol // Helper para derivar
    instancia extendida
```

```

10  static readonly tracker: unique symbol          // Metadata de rastreo
    de contexto
11  static readonly resolveConfig: unique symbol    // Helper de resolución
    de intercept-config
12  }

```

Fiber

Instancia de plugin en runtime.

```

1  fiber.uid: number | null          // ID único; 0 root, null si disposed
2  fiber.ctx: Context                // Contexto de ejecución del plugin
3  fiber.config: any                 // Configuración validada
4  fiber.state                       // Estado del ciclo de vida (emite '
    internal/status')
5  fiber.dispose(): Promise<void>    // Descargar plugin y ejecutar
    cleanup
6  fiber.name: string                // Nombre display (hereda del
    ancestro, o 'root')
7  fiber.store: Dict<Impl> | undefined // Snapshot de servicios requeridos
    (mientras cargado)
8  fiber.inertia: Promise<void> | undefined // Transición load/unload en
    vuelo

```

Máquina de estados

```

1  PENDING      LOADING      ACTIVE      UNLOADING      DISPOSED
2              FAILED

```

Métodos adicionales

```

1  fiber.assertActive()              // Lanza CordisError si ya disposed
2  fiber.getEffects(): EffectMeta[]  // Árbol de efectos activos
    etiquetados
3  fiber.await(): Promise<Fiber>     // Espera fin de trabajo lifecycle;
    relanza errores de startup
4  fiber.restart(): Promise<void>    // Dispose + recarga inmediata con la
    config actual
5  fiber.update(config: any, noSave?: boolean): any // Valida, aplica y
    reinicia

```

Efectos

```

1  ctx.effect(execute: () => SyncEffect, label?: string): Disposable<
    Promise<void>>
2  ctx.effect(execute: () => Effect, label?: string): AsyncDisposable<
    Promise<void>>
3
4  // Tipos
5  type Effect<T = any> = SyncEffect<T> | AsyncEffect<T>
6  type Disposable<T = any> = () => T
7
8  // EffectMeta (diagnóstico)
9  interface EffectMeta { label: string, children: EffectMeta[] }

```

Manejo de errores

```

1  class CordisError extends Error {
2      constructor(public code: CordisError.Code, message?: string)
3  }
4  // Códigos: { INACTIVE_EFFECT: 'cannot create effect on inactive
    context' }
5
6  class ValidationError extends TypeError {
7      name = 'ValidationError'
8      constructor(issues: readonly StandardSchemaV1.Issue[])
9  }

```

Plugin

```

1  ctx.plugin<P extends Plugin>(plugin: P, ...args: Spread<GetPluginConfig
    <P>>): Fiber & PromiseLike<Fiber>
2  ctx.inject(deps: Inject, callback: Plugin.Function<void>): Fiber &
    PromiseLike<Fiber>
3
4  // Formas de Plugin
5  type Plugin<T> = Plugin.Function<T> | Plugin.Constructor<T> | Plugin.
    Object<T>
6  // Plugin.Function<T>: (ctx: Context, config: T) => any (con metadata
    Base)
7  // Plugin.Constructor<T>: new (ctx: Context, config: T) => any
8  // Plugin.Object<T>: { apply(ctx: Context, config: T): any }
9  // Base: { name?, Config?, inject?, provide?, intercept? }
10

```

```

11 // Inject
12 type Inject = (keyof M)[] | { [K in keyof M]?: M[K] }

```

Herramientas (harness)

```

1 ctx.tools.register(toolDefinition): Disposer
2 ctx.tools.restrict(scopedRegistration): void
3 ctx.tools.guard(exec => boolean): void
4 ctx.tools.execute(params: {
5   callId: ToolCallId
6   name: string
7   arguments: Record<string, unknown>
8   signal: AbortSignal
9 }): Promise<{ content: ContentBlock[] }>

```

Servicios core principales

4.		
Servicio	Dueño	Rol
ctx.sessions	session	Almacén de sesiones en memoria, feed de eventos append-only
ctx.tools	tools	Registro de herramientas + pipeline de ejecución custodiado
ctx.agents	agent	Fábrica de agentes (create/resume), handles vivos
ctx.systemPrompt	system-prompt	Ensamblador de system prompt por paso
ctx.agentLoop	agent-loop	Driver concreto del bucle agente
ctx.storageDomain	storage-domain	Instalación tipada sobre backends de almacenamiento
ctx.workspaceRegistry	workspace	Registro de entidades Workspace con ID de marca
ctx.invariants	invariants	Checks invariantes con atribución de paquete
ctx.planMode	plan-mode	Estado de colaboración plan/mode plegado del log
ctx.commands	commands	Comandos directos del usuario sin pasar por modelo

Seams reemplazables

```

1 // Seam ctx.llm      backends múltiples
2 // Dueño: llm
3 // Implementaciones: llm-deepseek, llm-pi-ai, llm-replay
4 // Consumidores: agent-loop, compaction-basic
5
6 // Seam ctx.fs        proveedor de filesystem
7 // Dueño: fs

```

```
8 // Implementaciones: fs-local, fs-sandbox, fs-e2b
9 // Consumidores: tool-fs (+ fs-observation-policy vía gate de eventos)
```

Eventos clave del ciclo de vida del agente

Evento	Modo	Dispatcher	Listeners principales
agent/created	emit	agent	agent-presets, file-reference-local, goal-
agent/disposed	emit	agent	agent-loop, file-reference-local, goal-rou
agent/session-start	emit	agent-loop	agent-team, goal, goal-round-driver, hooks-
agent/status	emit	agent-loop	agent, agent-team, compaction-basic, goal-r
agent/pre-step	waterfall	agent-loop	agent, agent-instructions, compaction-basi
agent/request	waterfall	agent-loop	agent, webhook
agent/request-error	waterfall	agent-loop	compaction-basic, llm-retry
agent/turn-stopping	serial	agent-loop	hooks-claude-code, hooks-codex
agent/error	emit	agent-loop	acp, goal-round-driver, session-controller
agent/assistant-stream	emit	agent-loop	headless, session-controller
agent/inbox/inserted	emit	agent-loop	goal-round-driver
agent/inbox/claimed	emit	agent-loop	acp, goal-round-driver, subagent, tool-jobs
agent/inbox/discarded	emit	agent-loop	goal-round-driver, subagent
agent-loop/config-start-failed	emit	agent-loop	-

Eventos de sesión

Evento	Modo	Dispatcher	Listeners principales
session/created	emit	session	compaction, goal, hook-protocol, llm-retry, permission-preset
session/disposed	emit	session	agent-loop, agent-team, file-upload, session-controller, sess
session/event	emit	session	acp, agent-instructions, agent-loop, agent-presets, agent-tea
session/flush	parallel	session	session-persistence-jsonl, session-telemetry

Eventos de herramientas

Evento	Modo	Dispatcher	Listeners principales
tools/pre-execute	waterfall	tools	hooks-claude-code, hooks-codex, tool-jobs
tools/execute	waterfall	tools	session-checkpoint-policy, timeout-policy
tools/post-execute	waterfall	tools	hooks-claude-code, hooks-codex, repeat-to
tools/ptc-dispatch-log	waterfall	tools	spill-policy
tools/result	emit	tools	agent-instructions, subagent-in-process
tools/change	emit	agent-presets, tools	tool-subagent

Eventos de subagente

Evento	Modo	Dispatcher	Listeners
subagent/start	emit	subagent	hooks-claude-code, subagent
subagent/end	emit	subagent	hooks-claude-code, server, subagent
subagent/provider-added	emit	subagent	-

Eventos internos no declarados

Event string	Dispatchers	Listeners
internal/dispatch	-	agent-team, commands, compaction, fs, goal, goal-round-driver, hook-pro
internal/plugin	-	inspector, loader, lsp-stdio, modules
internal/service	-	agent-presets, gateway
internal/status	-	agent, inspector

configuracion

Configuración del proyecto dsh

Perfiles y arranque

Composición de la configuración

La configuración efectiva se construye aplicando capas en orden:

1. Parches de los bundles listados en `dsh.profile.bundles` del manifiesto del perfil.

2. `cordis.patch.yml` del propio perfil.
3. `DSH_HOME/cordis.patch.yml` (preferencias locales de máquina, mayor prioridad que el perfil). Cada `overlay-patch`

Las capas posteriores sobrescriben filas completas (no se hace merge profundo de config). Se pueden insertar nuevas filas.

Recarga de parches

- 4. `dsh.profile.patchReload: "live"` (por defecto en perfiles personalizados): observa cambios en archivos y reaplica.
- `dsh.profile.patchReload: "startup"`: carga única al iniciar.

Resolución de bundles y plugins

- Bundles internos (@deepseek-ai/dsh-base, -web-app, -headless, -sdk-app, -sdk-minimal, -acp-app) se resuelven desde la instalación de dsh.
- Bundles externos desde `node_modules` del perfil (gestionado por pnpm).
- Plugins con nombre simple se resuelven recorriendo directorios padre hasta `DSH_HOME/profiles/node_modules`.

Perfiles predefinidos

Se autoinician desde plantillas en el primer uso:

Perfil	Composición	patchReload
web	base + web-app	live
headless	base + headless	startup
sdk	base + sdk-app	startup
sdk-minimal	bundle independiente	startup
acp	base + acp-app	startup

Perfil faltante no predefinido → error con sugerencia: `dsh plugin -profile <name> add <package>`.

Argumentos de aplicación

- Flags del launcher terminan en el primer token no reconocido.
- El resto se pasa a `ctx.cmdlineArgs` para que los plugins de la app lo parseen.
- `dsh -profile web -port 8080` → `-port` va a la app web.
- `dsh -profile web -help` → imprime ayuda de la app y no arranca.
- `dsh -help` → ayuda del launcher.
- `-V/-version` antes de args de app → versión del launcher.

Precedencia de flags vs. configuración

- Un plugin inyecta `cmdlineArgs`, parsea argumentos y expone un servicio.

- Las filas de configuración usan expresiones como `port: !!js ctx.webStartup.port ?? 3080`.
- El flag tiene prioridad sobre el valor literal en la config.
- Si un parche de usuario reemplaza `config` con literales, se pierde la lectura en runtime.
- En modo `live`, editar un archivo de parche reevalúa expresiones pero no reinicia servicios (no puede cambiar un puerto ya servido).

Límite de argumentos

- `-` es consumido por el parser del launcher. Para pasar `-` literal a la app: `-- -`.
- Si el primer argumento de app es `web` o `plugin`, se selecciona ese subcomando.
- `ctx.cmdlineArgs.get()` es lectura inmutable compartida.

Apps embarcadas y sus argumentos

Perfil	Argumentos
<code>web</code>	<code>-host</code> , <code>-port</code> , <code>-trusted-host</code> (repetible), <code>-no-open</code>
<code>headless</code>	texto de la tarea como argumento posicional
<code>sdk</code>	sin opciones; protocolo JSON-RPC por stdio
<code>sdk-minimal</code>	sin opciones; mismo protocolo JSON-RPC
<code>acp</code>	sin opciones; Agent Client Protocol por stdio

Modo headless (one-shot)

```
1 dsh --profile headless "run the tests"
```

- Crea un Agent persistente, envía la tarea, espera quiescencia, flushea la sesión.
- Último texto no vacío del asistente → `stdout`.
- Razón final `turn/end` y deltas de razonamiento no vacíos → `stderr` (con encabezado `dsh: reasoning:`).
- Exit 0 si `completed`, 1 en otro caso.
- Sin tarea → error de uso.
- No monta servidor HTTP, WebSocket, ni navegador; no abre puertos.

Volcado de configuración

```
1 dsh --profile web --dump-default-config # solo capas de bundles
2 dsh --profile web --patch ./extra.yml --dump-config # incluye parches
```

- Imprime comentarios con archivo de origen y overlays que modificaron cada fila.
- Expresiones `!!js` sin evaluar.

- Nombres de plugin relativos se resuelven junto al archivo de parche.
- Destinos de parche no encontrados → stderr.
- No ejecuta providers de línea de comandos de la app; rechaza si hay argumentos de app.

Gestión de plugins (dsh plugin)

```
1 dsh plugin --profile <name> <args...>
```

- Si el perfil no existe, lo inicializa (plantilla predefinida o solo @deepseek-ai/dsh-base).
- Reenvía <args...> a pnpm con el directorio del perfil como working directory.
- pnpm debe estar en el PATH.
- Especificaciones de ruta relativas (., ../plugin, file:, link:) se anclan al directorio de invocación.
- Tras cada ejecución exitosa, reconcilia dsh.profile.bundles:
- Dependencia con "dsh": "bundle": "patch": "./cordis.patch.yml" → se añade a la pila de capas.
- Dependencia sin declaración de bundle → warning único.
- Dependencia eliminada → se quita de la pila.

Subagents Codex y Claude Code (bundles opcionales)

```
1 dsh plugin --profile <name> add @deepseek-ai/dsh-subagent-codex
2 dsh plugin --profile <name> add @deepseek-ai/dsh-subagent-claude-code
3 dsh plugin --profile <name> add @deepseek-ai/dsh-subagent-codex
  @deepseek-ai/dsh-subagent-claude-code
4 dsh plugin --profile <name> remove @deepseek-ai/dsh-subagent-codex
5 dsh plugin --profile <name> remove @deepseek-ai/dsh-subagent-claude-
  code
```

- La operación de pnpm cambia manifiesto y lista de bundles en disco.
- Un perfil en ejecución mantiene el conjunto de bundles de su inicio actual.
- **Reiniciar el perfil** después de añadir/quitar/actualizar bundles.
- Ediciones en cordis.patch.yml del perfil o home se aplican por hot reload.

Plugins desde Git

```
1 dsh plugin --profile tui add github:deepseek-harness/turtle-ui
2 dsh plugin --profile tui remove turtle-ui
3 dsh --profile tui
```

- Plugins con fuentes necesitan prepare script → pnpm 10 bloquea hasta permitir en pnpm-workspace.yaml.
- Primera ejecución de add falla con hint; copiar la clave impresa en pnpm-workspace.yaml y re-ejecutar.
- Tarballs preconstruidos o checkouts locales no requieren permisos.

Alias web (dsh web)

Alias fijo para --profile web. Los flags después de web pertenecen a la app web.

```
1 dsh web
2 dsh web --no-open
3 dsh web --patch ./extra.cordis.yml
4 dsh web --dump-config
5 dsh web --help
```

Flags de la app web

- -host, -port: sobrescriben valores compuestos.
- -trusted-host (repetible): añade autoridades a ctx.webRuntime.trustedHosts.
- -no-open: deshabilita apertura de navegador.

Comportamiento del servidor

- Por defecto sirve en http://127.0.0.1:3080.
- Solo abre navegador tras asentarse el árbol Loader completo.
- SSH_CONNECTION o SSH_TTY no vacíos → suprime handoff del navegador (imprime URL).
- No soporta -host 0.0.0.0 (error de uso).
- HMR de cliente siempre montado, inactivo hasta que pnpm run dev:web reconstruya bundles.

cordis.yml

```
1 # cordis.yml: lista de entradas. Cada entrada acepta:
2 - id: identidad estable para diff del loader
3 - name: ruta al módulo o nombre de paquete
4 - disabled: true # conserva la entrada pero no monta el plugin
5 - config: bloque validado por el schema del plugin
6 - !!js: solo dentro de config y disabled; evalúa expresión JS en tiempo
    de carga
```

- **Grupos:** anidan sub-listas de entradas; isolate da a un grupo su propia instancia de un servicio.
- **HMR:** @deepseek-ai/cordis-plugin-hmr vigila archivos y recarga plugins al guardar. Requiere @deepseek-ai/cordis-plugin-logger-console y @deepseek-ai/cordis-plugin-timer como dependencias.

Interfaces de configuración de plugins

@deepseek-ai/dsh-settings-file

```
1 interface Config {
2   path?: string; // Ruta al archivo. Default: 'settings.yaml' en
   'dshHome'.
3   dshHome?: string; // Directorio home del harness. Default: '
   $DSH_HOME' o '~/.dsh'.
4   watch?: boolean; // Observar cambios y recargar. Default: 'true'.
5   debounceMs?: number; // Ventana de estabilización del watcher (ms).
   Default: '100'.
6 }
```

@deepseek-ai/dsh-shell-env

```
1 interface Config {
2   dshHome?: string; // Default: '$DSH_HOME' o '~/.dsh'.
3 }
```

@deepseek-ai/dsh-skill

```
1 interface Config {
2   collectCacheMaxEntries?: number; // Máximo de catálogos completados
   en memoria.
3 }
```

@deepseek-ai/dsh-skill-filesystem

Requiere: skills

```
1 interface Config {
2     providerName?: string;           // Default: 'filesystem'.
3     includeDefaultRoots?: boolean;
4     dshHome?: string;                // Default: '$DSH_HOME' o '~/.dsh'.
5     agentsHome?: string;             // Default: '$DSH_AGENTS_HOME' o
        '~/.agents'.
6     customSkillDirs?: string[];
7     watch?: boolean;
8     watchUsePolling?: boolean;
9     watchStabilityThresholdMs?: number;
10    watchPollIntervalMs?: number;
11    watchMaxProjects?: number;
12    watchFollowSymlinks?: boolean;
13    bundledSkillDir?: string;         // Default: '$DSH_BUNDLED_SKILL_DIR'
        '.
14 }
```

@deepseek-ai/dsh-spill-local

```
1 interface Config {
2     root?: string;                  // Directorio raíz. Default: directorio
        temporal privado (0700).
3     cleanupPeriodDays?: number; // Días para limpieza de archivos viejos.
        Default: '30'. '0' desactiva.
4 }
```

@deepseek-ai/dsh-spill-policy

Requiere: tools

```
1 interface Config {
2     maxInlineBytes?: number; // Bytes máx. en texto plano antes de spill.
        Omitir desactiva la política.
3 }
```

@deepseek-ai/dsh-storage-domain

Requiere: storage

```
1 interface Config {
2     backend: string;                // Backend por defecto (requerido).
3     routes?: Record<string, string>; // Overrides por dominio: nombre ->
        backend.
4 }
```

@deepseek-ai/dsh-storage-json

Requiere: storage

```
1 interface Config {
2   root: string; // Directorio con archivos '<unit>.json' (requerido,
3   sin default).
4 }
```

@deepseek-ai/dsh-storage-sqlite

Requiere: storage

```
1 type JournalMode = 'wal' | 'delete' | 'truncate' | 'persist';
2
3 interface Config {
4   path: string; // Ruta al archivo de BD. Usar ':memory'
5   : ' para tests.
6   journalMode?: JournalMode; // Default: 'wal'.
7 }
```

@deepseek-ai/dsh-subagent-acp

Requiere: subagents, subprocess

```
1 type PermissionPolicy = 'allow' | 'reject';
2
3 interface Config {
4   providerName: string; // Default: 'acp'.
5   command: string; // Ejecutable del proceso hijo.
6   args: string[]; // Argumentos para el comando.
7   cwd?: string; // Directorio de trabajo. Default:
8   hereda del padre.
9   permission: PermissionPolicy; // 'reject' (default) o 'allow'.
10  env: Record<string, string>; // Variables de entorno extra.
11  disposeEofGraceMs?: number;
12  disposeGraceMs?: number;
13 }
```

@deepseek-ai/dsh-subagent-claude-code

Requiere: subagents, subprocess

```
1 interface Config {
2   providerName?: string; // Default: 'claude-code'.
3   model?: string;
4   env?: Record<string, string>;
5   permissionMode?: ClaudeCodePermissionMode;
6   disposeGraceMs?: number;
7 }
```

@deepseek-ai/dsh-subagent-codex

Requiere: subagents, subprocess

```
1 type CodexPermissionMode = 'never' | 'approve-for-me' | 'dangerously-bypass-approvals-and-sandbox';
2
3 interface Config {
4   providerName?: string; // Default: 'codex'.
5   model?: string;
6   env?: Record<string, string>;
7   permissionMode?: CodexPermissionMode;
8   disposeGraceMs?: number;
9 }
```

@deepseek-ai/dsh-subagent-dsh-sdk

Requiere: subagents

```
1 interface Config {
2   providerName: string; // Default: 'dsh-sdk'.
3   dshBin?: string;
4   profile: string; // Perfil del hijo (default: 'sdk').
5   patches: string[];
6   dshHome: string; // Home aislado para el hijo.
7   cwd?: string;
8   provider: string; // Default: 'deepseek-official'.
9   model: string; // Default: 'deepseek-v4-flash'.
10  maxTokens?: number;
11  env: Record<string, string>;
12  shutdownTimeoutMs?: number;
13  disposeEofGraceMs?: number;
14  disposeGraceMs?: number;
15 }
```

@deepseek-ai/dsh-subagent-fork-in-process

Requiere: subagents

```
1 interface Config {
2   providerName: string; // Default: 'fork'.
3 }
```

@deepseek-ai/dsh-subagent-spawn-in-process

Requiere: subagents

```
1 interface Config {
2   providerName: string; // Default: 'spawn'.
3 }
```

@deepseek-ai/dsh-subprocess-e2b

Requiere: e2b

```
1 interface Config {
2   pollMs?: number; // Cadencia de sondeo de estado (ms).
3 }
```

@deepseek-ai/dsh-system-prompt

```
1 interface Config {
2   includeHarnessIdentity?: boolean; // Default: 'true'.
3   includeRuntimeContext?: boolean; // Default: 'true'.
4   personaPrefix?: string; // Plantilla de prefijo de persona.
5   personaSuffix?: string; // Plantilla de sufijo de persona.
6   toolOrder?: string[]; // Orden de herramientas. Default:
   lexicográfico.
7 }
```

@deepseek-ai/dsh-terminal-bash

Requiere: terminals, sandboxPolicy, sessionProjections, subprocess

```
1 type ShellDialect = 'bash' | 'pwsh';
2
3 interface Config {
4   backendType?: string; // Default: 'shell'.
5   shellDialect?: ShellDialect; // Default: 'bash'.
6   shellPath?: string; // Default: '/bin/bash'.
7   shellArgs?: string[]; // Default: '--noprofile --norc -i'.
8   rows?: number;
9   cols?: number;
10  scrollbarLines?: number;
11  scrollbarMaxBytes?: number;
12  maxReadBytes?: number;
13  pollIntervalMs?: number;
14  exactProbeAfterMs?: number;
15  idleSilenceMs?: number;
16  handoffGraceMs?: number;
17  timeoutMs?: number;
18  disposeGraceMs?: number;
19 }
```

@deepseek-ai/dsh-time-context

Requiere: agents, sessionProjections

```
1 interface Config {
2   timeZone?: string;           // Zona horaria. Default: zona del
    proceso.
3   refreshIntervalMs?: number; // Mínimo ms entre inyecciones. '0' =
    cada paso.
4 }
```

@deepseek-ai/dsh-typer-loader

```
1 interface Config {
2   packages?: string[]; // Paquetes npm adicionales que deben resolver y
    exportar './typer'.
3 }
```

@deepseek-ai/dsh-user-approval

```
1 type ApprovalPolicy = 'ask' | 'never'
2
3 interface Config {
4   readonly policy?: ApprovalPolicy
5   // 'ask' delega en los answerers compuestos (fail-closed si no hay).
6   // 'never' rechaza automáticamente sin preguntar (CI/unattended).
7 }
```

@deepseek-ai/dsh-web

```
1 interface WebRuntimeConfig {
2   readonly searchProvider?: string // ID explícito del proveedor de bú
    squeda.
3   readonly fetchProvider?: string  // ID explícito del proveedor de
    fetch.
4 }
```

@deepseek-ai/dsh-web-app

```
1 interface Config {
2   openBrowser: boolean
3   printUrl: boolean
4   surfaceContext: boolean
5   trustedHosts: string[]
6 }
```

@deepseek-ai/dsh-web-fetch-http

```
1 interface Config {
2   maxResponseBytes?: number
3   maxBodyChars?: number
4   timeoutMs?: number
5   maxRedirects?: number
6   userAgent?: string
7 }
```

@deepseek-ai/dsh-web-search-deepseek

```
1 interface Config {
2   apiKey?: string
3   apiKeyEnv?: string           // Default: 'DEEPSEEK_API_KEY'
4   baseUrl?: string
5   model?: string              // Default: 'deepseek-v4-flash'
6   apiVersion?: string         // Default: '2023-06-01'
7   maxTokens?: number          // Default: 4096
8   maxUses?: number            // Default: 5
9 }
```

@deepseek-ai/dsh-web-search-exa

```
1 interface Config {
2   apiKey?: string
3   baseUrl?: string
4   searchType?: 'auto' | 'keyword' | 'neural' // Default: 'auto'
5   numResults?: number
6   highlightsPerResult?: number // Default: 1
7 }
```

@deepseek-ai/dsh-web-search-perplexity

```
1 interface Config {
2   apiKey?: string
3   baseUrl?: string
4   model?: string           // Default: 'sonar'
5   maxTokens?: number       // Default: 1024
6   searchRecency?: 'day' | 'week' | 'month' | 'year'
7 }
```

@deepseek-ai/dsh-webhook-github

```
1 interface Config {
2   readonly source: string
3   readonly path: string
4   readonly secretEnv: string
5   readonly maxBodyBytes: number
6 }
```

@deepseek-ai/dsh-workflow-worker-thread

```
1 interface Config {
2   provider?: string           // Default: 'spawn'
3   maxConcurrentAgents?: number // '0' auto-resuelve a 'min(16, max(1,
    cores - 2))'
4   maxTotalAgents?: number     // Default: 1000
5   maxItemsPerCall?: number    // Default: 4096
6   syncTimeoutMs?: number      // Default: 5000 ms
7   disposeGraceMs?: number     // Default: 5000 ms
8 }
```

@deepseek-ai/dsh-experimental-agent-team

```
1 interface Config {
2   readonly maxMembers?: number
3   readonly maxTasks?: number
4   readonly maxPendingMessagesPerMember?: number
5   readonly maxMessageBytes?: number
6   readonly disposalTimeoutMs?: number
7 }
```

@deepseek-ai/dsh-experimental-code-runtime-python

```
1 interface Config {
2   cpuSeconds?: number
3   maxWallMs?: number
4   addressSpaceMb?: number
5   maxLogBytes?: number
6   maxValueBytes?: number
7   graceMs?: number
8   pythonBin?: string
9 }
```

@deepseek-ai/dsh-experimental-inspector

```
1 interface Config extends Omit<InspectorOptions, 'clientOrigins'> {
2   clientOrigins?: string[]
3 }
4
5 interface InspectorOptions {
6   readonly host?: '127.0.0.1'
7   readonly port?: number
8   readonly clientOrigins?: readonly string[]
9   readonly captureFetch?: boolean
10  readonly maxRequestBodyBytes?: number
11  readonly maxResponseBodyBytes?: number
12  readonly maxBodyChunkBytes?: number
13  readonly maxJournalBytes?: number
14  readonly maxRetainedRequests?: number
15  readonly maxSourceFrameBytes?: number
16  readonly maxSourceRecordsPerFrame?: number
17  readonly maxQueuedRecords?: number
18  readonly maxQueuedBytes?: number
19  readonly startupTimeoutMs?: number
20  readonly stopTimeoutMs?: number
21  readonly clientReconnectBaseMs?: number
22  readonly clientReconnectMaxMs?: number
23  readonly clientRuntimeTimeoutMs?: number
24  readonly queryTimeoutMs?: number
25  readonly maxClientRuntimeObjects?: number
26  readonly maxClientRuntimeProperties?: number
27  readonly maxClientSourceBytes?: number
28  readonly maxCordisNodes?: number
29  readonly maxDisconnectedCordisTrees?: number
30 }
```

@deepseek-ai/dsh-experimental-tool-agent-team

```
1 interface Config {
2   readonly freshProvider?: string
3   readonly forkProvider?: string
4 }
```

@deepseek-ai/dsh-file-reference-local

```
1 interface Config {
2   maxResults?: number
3   maxEntries?: number
4   excludedDirectories?: string[]
5 }
```

@deepseek-ai/dsh-fs-local

```
1 interface Config {
2   cwd?: string
3   diffBasisMaxBytes?: number // Default: 10 MiB
4 }
```

@deepseek-ai/dsh-fs-sandbox

```
1 type Config = LocalConfig; // Misma configuración que fs-local
```

@deepseek-ai/dsh-goal

```
1 interface Config {
2   defaultMaxGoalRounds?: number
3 }
```

@deepseek-ai/dsh-headless

```
1 interface Config {
2   task: string
3 }
```

@deepseek-ai/dsh-hooks-claude-code

```
1 interface Config {
2   configPath: string
3   pluginRoot?: string
4   projectDir?: string
5   defaultTimeoutMs?: number // Default: 600000
6   stderrSummaryMaxChars?: number
7 }
```

@deepseek-ai/dsh-hooks-codex

```
1 interface Config {
2   configPath: string
3   model?: string
4   defaultTimeoutMs?: number // Default: 600000
5   stderrSummaryMaxChars?: number
6 }
```

@deepseek-ai/dsh-host-directory-picker-browse

```
1 interface Config {
2   maxEntries: number
3 }
```

@deepseek-ai/dsh-host-frontend-static

```
1 interface Config {
2   distIndex: string
3 }
```

@deepseek-ai/dsh-host-open-in-app

```
1 interface Config {
2   readonly probeTimeoutMs: number
3   readonly iconTimeoutMs: number
4   readonly launchWatchMs: number
5 }
```

@deepseek-ai/dsh-host-webserver

```
1 interface Config {
2   host: '127.0.0.1' | '0.0.0.0'
3   port: number
4   compression?: 'none' | 'gzip' // Default: 'none'
5   compressionLevel?: number // Default: 1
6   compressionThresholdBytes?: number // Default: 1024
7 }
```

@deepseek-ai/dsh-invariants

```
1 interface Config {
2   readonly enabled?: boolean // Default: true
3   readonly package_allowlist?: string[]
4   readonly package_blocklist?: string[]
5 }
```

@deepseek-ai/dsh-jobs-local

```
1 interface Config {
2   maxConcurrentJobsPerOwner?: number // Default: 10
3 }
```

@deepseek-ai/dsh-agent-loop

```
1 interface Config {
2   maxParallelToolCalls?: number
3   agents?: Array<{
4     id: string
5     provider: string
6     model: string
7     cwd?: string
8     sessionId?: string
9     resumeSessionId?: string
10  }>
11 }
```

@deepseek-ai/dsh-compaction-basic

```
1 interface Config {
2   thresholdRatio?: number
3   retainRatio?: number
4   summarizationProvider?: string
5   summarizationModel?: string
6   maxTokens?: number
7 }
```

@deepseek-ai/dsh-bash-local

```
1 interface Config {
2   timeoutMs?: number
3   maxOutputBytes?: number
4   graceMs?: number
5 }
```

@deepseek-ai/dsh-code-runtime-worker-thread

```
1 interface Config {
2   computeMs?: number
3   maxWallMs?: number
4   maxOldGenerationSizeMb?: number
5 }
```

@deepseek-ai/dsh-attachment-local

```
1 interface Config {
2   maxImageBytes?: number
3   maxImagePixels?: number
4   normalizedImageMaxBytes?: number
5 }
```

```
5 }
```

@deepseek-ai/dsh-credentials-local

```
1 interface Config {
2   path?: string
3   dshHome?: string
4   watch?: boolean
5   debounceMs?: number
6 }
```

Interfaces de configuración adicionales

ReplayProviderConfig

```
1 interface ReplayProviderConfig {
2   id: string
3   name?: string
4   models?: ReplayModelConfig[]
5   retryPolicy?: RetryPolicyConfig
6 }
```

ReplayModelConfig

```
1 interface ReplayModelConfig {
2   id: string
3   name?: string
4   description?: string
5   contextWindow?: number
6   inputModalities?: readonly ModelModality[]
7   defaultMaxTokens?: number
8   imageRequestTokens?: number
9   reasoningEfforts?: string[]
10  defaultReasoningEffort?: string
11 }
```

LspLocalServerConfig

```
1 interface LspLocalServerConfig {
2   command: string
3   extensionToLanguage: Record<string, string>
4   args?: string[]
5   env?: Record<string, string>
6   initializationOptions?: unknown
```

```
7   configuration?: unknown
8   maxMessageBytes?: number
9   maxStderrBytes?: number
10  maxDocumentBytes?: number
11  shutdownTimeoutMs?: number
12  killGraceMs?: number
13 }
```

StdioConfig

```
1 interface StdioConfig {
2   transport: 'stdio'
3   serverName: string
4   command: string
5   args: string[]
6   env: Record<string, string>
7   cwd: string
8   toolCallTimeoutMs: number
9   failOnStartupError: boolean
10  reconnect?: ReconnectConfig
11 }
```

StreamableHttpConfig

```
1 interface StreamableHttpConfig {
2   transport: 'streamable-http'
3   serverName: string
4   url: string
5   headers: Record<string, string>
6   toolCallTimeoutMs: number
7   failOnStartupError: boolean
8   reconnect?: ReconnectConfig
9 }
```

ReconnectConfig

```
1 interface ReconnectConfig {
2   enabled?: boolean
3   initialDelayMs?: number
4   maxDelayMs?: number
5   maxAttempts?: number
6 }
```

PresetSpec

```
1 interface PresetSpec {
2     sandbox: SandboxMode
3     approval: ApprovalPolicy
4     name?: string
5     description?: string
6 }
```

PlanModeConfig

```
1 interface PlanModeConfig {
2     section: string
3 }
```

JsonRpcConfig

```
1 interface JsonRpcConfig {
2     maxTokensAsSuccess?: boolean
3     input?: Readable
4     output?: Writable
5     exit?: (code: number) => void
6 }
```

SessionQueryConfig

```
1 interface Config extends SessionQueryConfig {
2     path: string
3     openAt?: OpenAt
4     journalMode?: JournalMode
5     defaultLimit?: number
6     maxLimit?: number
7     snippetChars?: number
8     persistedReadConcurrency?: number
9     preparedSessionCacheSize?: number
10 }
```

SessionTelemetryMode

```
1 enum SessionTelemetryMode {
2     FEEDBACK_ONLY = 'FEEDBACK_ONLY',
3     DISABLED = 'DISABLED',
4 }
```

ToolSubagentConfig

```
1 interface Config {
2   provider: string
3   toolName?: string
4   modelSelectionSettings?: boolean
5   enableRunInBackground?: boolean
6   backgroundMode?: 'one-shot' | 'continuable'
7   agentOptions?: AgentOptions
8   persona?: string
9   toolFilter?: { allow?: string[]; deny?: string[] }
10  maxDepth?: number | 'provider-managed'
11 }
```

ToolPresentationMode

```
1 type ToolPresentationMode = 'native' | 'ptc' | 'both'
```

Variables de entorno

- *DSH_HOME* : directorio homedelharness.Default : ~/.dsh.

Comandos de desarrollo

```
1 pnpm dsh web                # Inicia el host desde fuente (usa el
   fallback SRC)
2 pnpm run dev:web            # Inicia el watcher de plugins del cliente
3 pnpm run build:lib          # Reconstruye los contratos estrictos de
   Typert (Host y Cliente)
4 pnpm run build:lib:host     # Genera artefactos estrictos del Host
5 pnpm run build:lib:client   # Genera artefactos estrictos del Cliente
```

ejemplos

Ejemplos de código real del repositorio

Ejemplo 1: Declaración de un servicio remoto (Host)

Fuente: patrón de TypertRemoteService en el repo

```
1 import type { Agent } from '@deepseek-ai/dsh-agent'
2 import { TypertRemoteService, Remote, RemoteScope } from '@deepseek-ai/
  dsh-typert-protocol'
3 import type { Context } from '@deepseek-ai/cordis'
4
5 export class GoalService extends TypertRemoteService {
6   constructor(ctx: Context) {
7     super(ctx, 'goals')
8   }
9
10  @Remote('create')
11  createForClient(
12    agent: Agent,
13    request: { objective: string },
14    signal: AbortSignal,
15  ): { accepted: boolean } {
16    signal.throwIfAborted()
17    return this.create(agent, request)
18  }
19
20  @RemoteScope('agent', 'current')
21  currentForClient(): { accepted: boolean } {
22    return { accepted: true }
23  }
24
25  private create(_agent: Agent, request: { objective: string }): {
26    accepted: boolean } {
27    return { accepted: request.objective.length > 0 }
28  }
```

Ejemplo 2: Consumo de un servicio remoto (Cliente)

Fuente: patrón de consumo remoto en el repo

```
1 import type { Context } from '@deepseek-ai/cordis'
2 import type {} from '@deepseek-ai/dsh-api-remotes/client'
3
4 export const inject = ['remote', 'remote.goals']
5
6 declare const ctx: Context
```

```
7 declare const agentId: string
8
9 // Llamada directa
10 await ctx.remote.goals.create(agentId, { objective: 'ship it' })
11
12 // Llamada con ámbito de agente
13 declare const agentCtx: any
14 await agentCtx.remote.goals.create({ objective: 'ship it' })
```

Ejemplo 3: Llamada remota con manejo de errores

Fuente: patrón de manejo de errores remotos en el repo

```
1 import type { Context } from '@deepseek-ai/cordis'
2 import { isRemoteFailure } from '@deepseek-ai/dsh-api-gateway/client'
3 import type {} from '@deepseek-ai/dsh-api-remotes/client'
4
5 export const inject = ['remote', 'remote.notes']
6
7 declare const ctx: Context
8
9 export async function noteTitles(): Promise<readonly string[]> {
10   const result = await ctx.remote.notes.list()
11   if (!result.ok) {
12     if (result.error.code === 'note/not-found') return []
13     throw result.error
14   }
15   return result.value.map(row => row.title)
16 }
17
18 export async function renderTitles(): Promise<string> {
19   try {
20     return (await noteTitles()).join(', ')
21   } catch (error: unknown) {
22     if (!isRemoteFailure(error)) throw error
23     return `unavailable (${error.code})`
24   }
25 }
26
27 export function hostLabel(): string {
28   const { home, isLoopback } = ctx.remote.$host
29   return home ?? (isLoopback ? 'local host' : 'remote host')
30 }
```

Ejemplo 4: Prueba de Owner con remoteErrorOf

Fuente: patrón de pruebas de owner en el repo

```
1 import { remoteErrorOf } from '@deepseek-ai/dsh-typer-protocol'
2 import { expect, it } from 'vitest'
3
4 declare function rename(noteId: string, title: string): Promise<void>
5
6 it('refuses an unknown note before writing', async () => {
7   const failure = await rename('n-404', 'fresh title').catch((error:
8     unknown) => error)
9   expect(remoteErrorOf(failure)).toMatchObject({
10     code: 'note/not-found',
11     details: { noteId: 'n-404' },
12   })
13 })
```

Ejemplo 5: Prueba de Cliente con TestRemote

Fuente: patrón de pruebas de cliente en el repo

```
1 import { Context } from '@deepseek-ai/cordis'
2 import { RemoteError, TestRemote } from '@deepseek-ai/dsh-client-test-
3   runtime'
4 import { expect, it } from 'vitest'
5
6 it('renders the failure code the Host reported', async () => {
7   const ctx = new Context()
8   const remote = new TestRemote(ctx, {
9     notes: {
10       list: () => Promise.resolve({
11         ok: false as const,
12         error: new RemoteError('note/not-found', 'no note "n-404"', {
13           noteId: 'n-404' }),
14       }),
15     },
16   })
17   remote.$host = { home: '/home/fixture', isLoopback: true }
18   await expect(ctx.remote.notes.list()).resolves.toMatchObject({ error:
19     { code: 'note/not-found' } })
20 })
```

Ejemplo 6: Herramienta del harness

Fuente: patrón de herramienta en el repo

```
1 // greet-tool.ts
2 import type { Context } from '@deepseek-ai/cordis'
3 import { brandString } from '@deepseek-ai/dsh-brand'
4 import { defineTool } from '@deepseek-ai/dsh-tools'
5 import type { ToolCallId } from '@deepseek-ai/dsh-llm'
6
7 export const name = 'greet-tool'
8 export const inject = ['tools']
9
10 export function apply(ctx: Context) {
11   ctx.tools.register(defineTool({
12     name: 'greet',
13     description: 'Greet the named person.',
14     parameters: {
15       name: { type: 'string', required: true, description: 'Who to greet' },
16     },
17     output: {
18       schema: { type: 'string' },
19       render: (_args, value) => [{ type: 'text', text: value }],
20     },
21     async execute(args) {
22       return `Hello, ${args.name}!`
23     },
24   }))
25
26   void (async () => {
27     const result = await ctx.tools.execute({
28       callId: brandString<ToolCallId>('demo-1'),
29       name: 'greet',
30       arguments: { name: 'Cordis' },
31       signal: new AbortController().signal,
32     })
33     console.log('tool replied:', JSON.stringify(result.content))
34   })()
35 }
```

```
1 # cordis.yml
2 - name: '@deepseek-ai/dsh-system-prompt'
3 - name: '@deepseek-ai/dsh-tools'
4 - name: './tool-logger.ts'
5 - name: './greet-tool.ts'
```

Ejemplo 7: Configuración de agente en cordis.yml

Fuente: configuración de @deepseek-ai/dsh-agent-loop

```
1 plugins:
2   '@deepseek-ai/dsh-agent-loop':
3     config:
4       maxParallelToolCalls: 4
5       agents:
6         - id: main-agent
7           provider: deepseek
8           model: deepseek-chat
9           cwd: /home/user/project
```

Ejemplo 8: Configuración de compactación básica

Fuente: configuración de @deepseek-ai/dsh-compaction-basic

```
1 plugins:
2   '@deepseek-ai/dsh-compaction-basic':
3     config:
4       thresholdRatio: 0.8
5       retainRatio: 0.2
6       summarizationProvider: deepseek
7       summarizationModel: deepseek-chat
8       maxTokens: 4096
```

Ejemplo 9: Servidor MCP por stdio

Fuente: configuración de @deepseek-ai/dsh-mcp-client

```
1 plugins:
2   '@deepseek-ai/dsh-mcp-client':
3     transport: stdio
4     serverName: my-server
5     command: node
6     args: ['server.js']
7     env: {}
8     cwd: /app
9     toolCallTimeoutMs: 30000
10    failOnStartupError: true
11    reconnect:
12      enabled: true
13      initialDelayMs: 500
```

```
14         maxDelayMs: 30000
15         maxAttempts: 10
```

Ejemplo 10: Subagente ACP

Fuente: configuración de @deepseek-ai/dsh-subagent-acp

```
1  plugins:
2    '@deepseek-ai/dsh-subagent-acp':
3      providerName: acp
4      command: /usr/local/bin/acp-agent
5      args: ['--config', 'agent.yaml']
6      permission: allow
7      env:
8        DEEPSEEK_API_KEY: sk-...
9      disposeEOFGraceMs: 5000
10     disposeGraceMs: 2000
```

Ejemplo 11: Herramienta subagente con filtro

Fuente: configuración de @deepseek-ai/dsh-tool-subagent

```
1  plugins:
2    '@deepseek-ai/dsh-tool-subagent':
3      provider: spawn
4      toolName: my-subagent
5      enableRunInBackground: true
6      backgroundMode: continuable
7      agentOptions:
8        model: deepseek-v4-flash
9      toolFilter:
10        allow: ['read_file', 'write_file']
11        deny: ['shell']
12      maxDepth: 2
```

Ejemplo 12: Presentación de herramientas en modo PTC

Fuente: configuración de @deepseek-ai/dsh-tools

```
1  plugins:
2    '@deepseek-ai/dsh-tools':
```

```
3     mode: ptc
4     maxParallelSubCalls: 5
```

Ejemplo 13: Configuración de Python runtime

Fuente: configuración de @deepseek-ai/dsh-experimental-code-runtime-python

```
1 # cordis.yml
2 '@deepseek-ai/dsh-experimental-code-runtime-python':
3   cpuSeconds: 10
4   maxWallMs: 30000
5   addressSpaceMb: 512
6   maxLogBytes: 1048576
7   maxValueBytes: 524288
8   pythonBin: /usr/bin/python3
```

Ejemplo 14: Configuración de headless

Fuente: configuración de @deepseek-ai/dsh-headless

```
1 '@deepseek-ai/dsh-headless':
2   task: "Analiza el archivo README.md y sugiere mejoras"
```

Ejemplo 15: package.json para Remote

Fuente: patrón de package.json para servicios remotos

```
1 {
2   "exports": {
3     "./typert": { "types": "./lib/typert.host.d.ts", "default": "./lib/typert.host.js" },
4     "./remote": { "types": "./lib/typert.remote-client.d.ts", "default": "./lib/typert.remote-client.js" }
5   },
6   "peerDependencies": { "@deepseek-ai/dsh-typert-protocol": "workspace:~" },
7   "devDependencies": { "@deepseek-ai/dsh-typert-protocol": "workspace:~" }
8 }
```

Ejemplo 16: Hook de permission gate

Fuente: patrón de hooks en el repo

```
1 export function apply(ctx: Context) {
2   ctx.on('tools/pre-execute', async (exec, next): Promise<
3     PreToolDecision> => {
4     if (!(await isAllowed(exec))) {
5       return { kind: 'deny', reason: 'Denied by policy.' }
6     }
7     return next()
8   })
9 }
```

Ejemplo 17: UI — streaming en tiempo real

Fuente: patrón de UI en el repo

```
1 export function apply(ctx: Context) {
2   ctx.on('agent/assistant-stream', ({ frame }) => {
3     if (frame.type === 'chunk' && frame.chunk.type === 'text-delta') {
4       render(frame.chunk.text)
5     }
6   })
7
8   onUserInput(text => ctx.agents.get(sessionId)?.followup(
9     createUserMessage({
10      content: [{ type: 'text', text }],
11      source: { kind: 'user' },
12    })))
13 }
```

Ejemplo 18: Protocol driver (stdio)

Fuente: patrón de protocol driver en el repo

```
1 export function apply(ctx: Context) {
2   ctx.on('session/event', (_session, event) => {
3     if (event.type === 'assistant/message' || event.type === 'assistant
4       /attempt') {
5       for (const { chunk } of expandAssistantStream(event.data.stream))
6         {
7           if (chunk.type === 'text-delta') { /* enviar a cliente */ }
8         }
9     }
10  }
```

```
7     }  
8   })  
9   // Entrada: crear/recuperar agente, alimentar con followup(),  
    devolver recibo de encolado  
10  // Desmontaje: AgentHandle.dispose() (stop + await exit)  
11 }
```

Ejemplo 19: Waterfall para decisiones

Fuente: patrón de waterfall en el repo

```
1 // Listener que envuelve:  
2 ctx.on('internal/update', function (config, next) {  
3   console.log('antes:', config)  
4   const result = next(config) // delega al siguiente listener / built  
    -in  
5   console.log('después:', result)  
6   return result // propaga (posiblemente modificado)  
7 })  
8  
9 // Política que veta:  
10 ctx.on('some-decision-event', function (payload, next) {  
11   if (payload.blocked) {  
12     return { denied: true } // cortocircuito no llama a next()  
13   }  
14   return next()  
15 })
```

Ejemplo 20: Configuración validada con Schemastery

Fuente: patrón de configuración en el repo

```
1 import Schema from '@deepseek-ai/schemastery'  
2  
3 export interface Config {  
4   greeting: string  
5   targets: string[]  
6 }  
7  
8 export const Config: Schema<Config> = Schema.object({  
9   greeting: Schema.string().default('Hello'),  
10  targets: Schema.array(String).default(['world']),  
11 })  
12
```

```
13 export function apply(ctx: Context, config: Config) {  
14   for (const target of config.targets) {  
15     console.log(`${config.greeting}, ${target}!`)  
16   }  
17 }
```

```
1 # cordis.yml  
2 - name: './config-demo.ts'  
3   config:  
4     targets: ['alpha', 'beta']
```

glosario

Glosario técnico del proyecto dsh

A

- **Append-only:** Propiedad de los logs de sesión: son inmutables y solo crecen. Los eventos durables (turn/, step/, user/message, assistant/message, assistant/attempt, tool/*) no se modifican ni eliminan.
- **Agent Client Protocol (ACP):** Protocolo de comunicación con agentes por stdio.
- **AgentHandle:** Handle vivo de un agente. Métodos: followup(), steer(), inject(), dispose().

B

- **Bundle:** Paquete que compone una aplicación concreta (ej. ctx.agentLoop). Los bundles internos son dsh-base, dsh-web-app, dsh-headless, dsh-sdk-app, dsh-sdk-minimal, dsh-acp-app.
- **Bail:** Modo de dispatch de eventos: secuencial síncrono, se detiene en el primer listener que retorna un valor.

C

- **Cordis:** Kernel de plugins del harness. Proporciona Context, Service, Fiber, eventos y ciclo de vida.
- **Core:** Servicio único, dueño del estado (ej. ctx.sessions, ctx.tools).

- **ctx:** Objeto central de Cordis: servicios, eventos y ciclo de vida accesibles a través de ctx.

D

- **Driver:** Orquestador del bucle del agente. Opera en 4 modos de ejecución en tiempo real.
- **Disposer:** Función cleanup retornada por un efecto. Se ejecuta en orden inverso de registro.
- **DispatchMode:** Tipo de dispatch de eventos: 'emit' | 'parallel' | 'serial' | 'bail' | 'waterfall'.

E

- **Emit:** Modo de dispatch de eventos: broadcast sin retorno, sin orden garantizado.
- **Effect:** Recurso externo registrado con `ctx.effect()`. El disposer se ejecuta al descargar el fiber.
- **EffectMeta:** Metadatos de diagnóstico de efectos: `label: string, children: EffectMeta[]`.

F

- **Fiber:** Instancia de plugin en runtime. Estados: PENDING → LOADING → ACTIVE → UNLOADING → DISPOSED (o FAILED).
- **FiberState:** Enum de estados del fiber.

G

- **Gateway:** Resuelve objetos complejos del Host (ej. Agent) usando `ctx.typepert.lookups`.

H

- **HMR (Hot Module Reload):** Recarga de plugins al guardar archivos. Requiere `cordis-plugin-hmr`.
- **Headless:** Modo de ejecución one-shot sin interfaz. `dsh -profile headless "task"`.

I

- **Inject:** Declaración de dependencias duras de un plugin. `inject: ['servicio1', 'servicio2']`.
- **Isolate:** Crear un hijo con ámbito de servicio independiente para un nombre.

J

- **JSON-RPC**: Protocolo RPC por stdio usado por los perfiles `sdk` y `sdk-minimal`.

L

- **Loader**: Cargador de plugins. Resuelve bundles y plugins desde la instalación de `dsh` o `node_modules` del perfil.
- **Live reload**: Modo de recarga de parches que observa cambios en archivos y reaplica (`patchReload: "live"`).

M

- **MCP (Model Context Protocol)**: Protocolo para conectar servidores de herramientas. Transportes: `stdio` y `streamable-http`.
- **Monotonic guard**: Guardia monotónica para denegación final en el pipeline de herramientas.

O

- **Overlay**: Capa de parche aplicada con `-patch <path>`. Las capas posteriores sobrescriben filas completas.

P

- **Parallel**: Modo de dispatch de eventos: ejecución concurrente de todos los listeners.
- **Patch**: Archivo YAML que modifica la configuración. Capas: `bundles` → `perfil` → `home` → `overlays`.
- **Perfil**: Composición de bundles y parches. Se inicia con `dsh -profile <nombre>`.
- **PTC (Prompt-to-Code)**: Modo de presentación de herramientas donde el código se genera como herramienta.
- **Plugin**: Unidad de extensión Cordis. Formas: función, clase con `apply`, u objeto `apply`.

R

- **Remote**: Decorador para exponer un método como endpoint RPC unario.
- **RemoteScope**: Decorador para métodos que operan sobre un contexto resuelto por ámbito (ej. `'agent'`).

- **RemoteError**: Error tipado para el cliente remoto. Códigos declarados en `RemoteErrorDetailsMap`.

S

- **Seam**: Interfaz abstracta (ej. `ctx.llm`, `ctx.fs`). Varios backends pueden registrarse.
- **Serial**: Modo de dispatch de eventos: ejecución uno a uno, espera cada listener.
- **Service**: Clase base para servicios en `ctx.<name>`. Registra la instancia en el contexto.
- **Schemastery**: Librería de validación de esquemas de configuración.
- **Settlement**: Frame de fin confirmado que consolida un stream como `assistant/message` o `assistant/attempt`.
- **Spill**: Desbordamiento de resultados de herramientas a archivos cuando exceden `maxInlineBytes`.
- **Subagent**: Agente hijo delegado. Proveedores: `spawn`, `fork`, `acp`, `codex`, `claude-code`, `dsh-sdk`.

T

- **Typert**: Protocolo RPC tipado entre Host y Cliente. Artefactos: `typert.host.js`, `typert.remote-client.js`.
- **TypertRemoteService**: Clase base para servicios de negocio expuestos al cliente.
- **Tool**: Herramienta registrada en `ctx.tools`. Pipeline: `pre-policy` → `monotonic guards` → `dispatch` → `post-policy` → `final-result`.
- **ToolCallId**: Identificador de llamada a herramienta con marca de tipo.

W

- **Waterfall**: Modo de dispatch de eventos: middleware encadenado. Cada listener recibe `(...args, next)`.
- **Workspace**: Entidad de espacio de trabajo con ID de marca, registrada en `ctx.workspaceRegistry`.

Y

- **YAML !!js**: Expresión JavaScript evaluada en tiempo de carga dentro de `config` y `disabled`.

patrones

Patrones de uso comunes

1. Envolver recurso externo en un efecto

```
1 export function apply(ctx: Context) {
2   ctx.effect(() => {
3     const timer = setInterval(() => console.log('tick'), 200)
4     return () => clearInterval(timer)    // disponer
5   })
6 }
```

Flujo:

1. apply se ejecuta al cargar el plugin.
2. ctx.effect ejecuta el callback inmediatamente.
3. El disponer retornado se registra para cleanup.
4. Al descargar el fiber, el disponer se ejecuta en orden inverso.

2. Montar un sub-plugin y desmontarlo luego

```
1 const fiber = ctx.plugin(heartbeat)
2 // ... más tarde
3 await fiber.dispose()
```

Flujo:

1. ctx.plugin carga el plugin y retorna un Fiber awaitable.
2. El fiber pasa por estados: PENDING → LOADING → ACTIVE.
3. dispose() ejecuta cleanup y pasa a DISPOSED.

3. Proveer un servicio

```
1 declare module '@deepseek-ai/cordis' {
2   interface Context { greeter: GreeterService }
3 }
4
5 export class GreeterService extends Service {
6   constructor(ctx: Context) { super(ctx, 'greeter') }
7   greet(who: string) { return 'Hello, ${who}!' }
8 }
9
10 export function apply(ctx: Context) { ctx.plugin(GreeterService) }
```

Flujo:

1. Declarar el servicio en la interfaz Context (module augmentation).
2. Extender Service y llamar super(ctx, 'greeter') en el constructor.
3. Registrar con ctx.plugin(GreeterService).
4. Otros plugins pueden consumir con inject: ['greeter'].

4. Consumir un servicio con dependencia dura

```
1 export const inject = ['greeter']
2 export function apply(ctx: Context) {
3   console.log(ctx.greeter.greet('world'))
4 }
```

Flujo:

1. Declarar inject con los nombres de servicios requeridos.
2. El plugin espera a que los servicios estén disponibles.
3. Si un servicio requerido cambia, el plugin se recarga automáticamente.

5. Consumir un servicio opcional

```
1 export function apply(ctx: Context) {
2   const greeter = ctx.get('greeter')
3   console.log(greeter?.greet('maybe') ?? 'no greeter available')
4 }
```

Flujo:

1. Usar `ctx.get('greeter')` sin declarar `inject`.
2. Si el servicio no existe, retorna `undefined`.
3. Manejar el caso con optional chaining y fallback.

6. Declarar y emitir eventos tipados

```
1 declare module '@deepseek-ai/cordis' {
2   interface Events {
3     'stats/report'(name: string, count: number): void
4   }
5 }
6
7 // Emisión
8 this.ctx.emit('stats/report', name, next)
9
10 // Escucha (efecto automático)
11 ctx.on('stats/report', (name, count) => {
12   console.log('[stats] ${name} -> ${count}')
```

Flujo:

1. Declarar el evento en la interfaz `Events` (module augmentation).
2. Emitir con `ctx.emit`.
3. Escuchar con `ctx.on` — el listener se registra como efecto y se limpia automáticamente.

7. Waterfall: transformar o cortocircuitar

```
1 declare module '@deepseek-ai/cordis' {
2   interface Events {
3     'demo/transform'(input: string, next: () => Promise<string>):
4       Promise<string>
5   }
6 }
7
8 // Listener que transforma
9 ctx.on('demo/transform', async (input, next) => {
10   const downstream = await next()
11   return downstream.toUpperCase()
12 })
```

```
11  })
12
13  // Listener que cortocircuita
14  ctx.on('demo/transform', async (input, next) => {
15    if (input.includes('blocked')) return '** blocked **'
16    return next()
17  })
18
19  // Invocación
20  const result = await ctx.waterfall('demo/transform', 'hello', async ()
    => 'hello')
```

Flujo:

1. Cada listener recibe (input, next).
2. next() invoca el siguiente listener en la cadena.
3. No llamar a next() cortocircuita la cadena.
4. El valor de retorno se propaga hacia afuera.

8. Configuración validada con Schemastery

```
1  import Schema from '@deepseek-ai/schemastery'
2
3  export interface Config {
4    greeting: string
5    targets: string[]
6  }
7
8  export const Config: Schema<Config> = Schema.object({
9    greeting: Schema.string().default('Hello'),
10    targets: Schema.array(String).default(['world']),
11  })
12
13  export function apply(ctx: Context, config: Config) {
14    for (const target of config.targets) {
15      console.log(`${config.greeting}, ${target}!`)
16    }
17  }
```

```
1  # cordis.yml
2  - name: './config-demo.ts'
3    config:
4      targets: ['alpha', 'beta']
```

Flujo:

1. Definir interfaz Config y schema Schemastery.
2. El loader valida la config del YAML contra el schema.
3. apply recibe la config validada.

9. Valores computados en YAML (!!js)

```
1 - name: './config-demo.ts'
2   config:
3     greeting: !!js process.env.DEMO_GREETING ?? 'Hello'
```

Flujo:

1. !!js evalúa la expresión JavaScript en tiempo de carga.
2. Solo permitido dentro de config y disabled.
3. En modo live, editar el archivo reevalúa expresiones.

10. Diagnóstico de plugins en PENDING

```
1 import { FiberState, type Context } from '@deepseek-ai/cordis'
2
3 export function apply(ctx: Context) {
4   setTimeout(() => {
5     for (const runtime of ctx.registry.values()) {
6       for (const fiber of runtime.fibers) {
7         if (fiber.state === FiberState.PENDING) {
8           console.log(`${fiber.name} is PENDING      a required service
9             is missing`)
10        }
11      }
12    }, 500)
13  }
```

Flujo:

1. Iterar ctx.registry.values() para obtener runtimes.
2. Iterar runtime.fibers para obtener fibers.
3. Verificar fiber.state === FiberState.PENDING.

11. Servicio remoto Typert (Host)

```
1 import type { Agent } from '@deepseek-ai/dsh-agent'
2 import { TypertRemoteService, Remote, RemoteScope } from '@deepseek-ai/
  dsh-typert-protocol'
3 import type { Context } from '@deepseek-ai/cordis'
4
5 export class GoalService extends TypertRemoteService {
6   constructor(ctx: Context) {
7     super(ctx, 'goals')
8   }
9
10  @Remote('create')
11  createForClient(
12    agent: Agent,
13    request: { objective: string },
14    signal: AbortSignal,
15  ): { accepted: boolean } {
16    signal.throwIfAborted()
17    return this.create(agent, request)
18  }
19
20  @RemoteScope('agent', 'current')
21  currentForClient(): { accepted: boolean } {
22    return { accepted: true }
23  }
24
25  private create(_agent: Agent, request: { objective: string }): {
26    accepted: boolean } {
27    return { accepted: request.objective.length > 0 }
28  }
```

Flujo:

1. Extender `TypertRemoteService` con namespace `'goals'`.
2. Decorar métodos públicos con `@Remote` o `@RemoteScope`.
3. El Gateway resuelve objetos complejos (ej. `Agent`) usando `ctx.typert.lookups`.

12. Consumo remoto con manejo de errores (Cliente)

```
1 import type { Context } from '@deepseek-ai/cordis'
2 import { isRemoteFailure } from '@deepseek-ai/dsh-api-gateway/client'
3 import type {} from '@deepseek-ai/dsh-api-remotes/client'
4
5 export const inject = ['remote', 'remote.notes']
6
7 declare const ctx: Context
8
9 export async function noteTitles(): Promise<readonly string[]> {
10   const result = await ctx.remote.notes.list()
11   if (!result.ok) {
12     if (result.error.code === 'note/not-found') return []
13     throw result.error
14   }
15   return result.value.map(row => row.title)
16 }
17
18 export async function renderTitles(): Promise<string> {
19   try {
20     return (await noteTitles()).join(', ')
21   } catch (error: unknown) {
22     if (!isRemoteFailure(error)) throw error
23     return `unavailable (${error.code})`
24   }
25 }
26
27 export function hostLabel(): string {
28   const { home, isLoopback } = ctx.remote.$host
29   return home ?? (isLoopback ? 'local host' : 'remote host')
30 }
```

Flujo:

1. Declarar inject: ['remote', 'remote.notes'].
2. Llamar ctx.remote.notes.list().
3. Verificar result.ok; si es false, ramificar por result.error.code.
4. Usar isRemoteFailure(error) para distinguir fallos remotos de defectos locales.

13. Hook de permission gate

```
1 export function apply(ctx: Context) {
2   ctx.on('tools/pre-execute', async (exec, next): Promise<
3     PreToolDecision> => {
4     if (!(await isAllowed(exec))) {
5       return { kind: 'deny', reason: 'Denied by policy.' }
6     }
7     return next()
8   })
9 }
```

Flujo:

1. Escuchar tools/pre-execute (waterfall).
2. Si la política deniega, retornar `kind: 'deny', reason` sin llamar a `next()`.
3. Si permite, llamar a `next()` para continuar la cadena.

14. UI: streaming en tiempo real

```
1 export function apply(ctx: Context) {
2   ctx.on('agent/assistant-stream', ({ frame }) => {
3     if (frame.type === 'chunk' && frame.chunk.type === 'text-delta') {
4       render(frame.chunk.text)
5     }
6   })
7
8   onUserInput(text => ctx.agents.get(sessionId)?.followup(
9     createUserMessage({
10      content: [{ type: 'text', text }],
11      source: { kind: 'user' },
12    })))
13 }
```

Flujo:

1. Escuchar agent/assistant-stream para chunks transitorios.
2. Enviar entrada de usuario con `ctx.agents.get(sessionId)?.followup(...)`.

15. Protocol driver (stdio)

```

1 export function apply(ctx: Context) {
2   ctx.on('session/event', (_session, event) => {
3     if (event.type === 'assistant/message' || event.type === 'assistant
      /attempt') {
4       for (const { chunk } of expandAssistantStream(event.data.stream))
5         {
6           if (chunk.type === 'text-delta') { /* enviar a cliente */ }
7         }
8     })
9     // Entrada: crear/recuperar agente, alimentar con followup(),
      devolver recibo de encolado
10    // Desmontaje: AgentHandle.dispose() (stop + await exit)
11  }

```

Flujo:

1. Escuchar session/event para eventos durables.
2. Expandir el stream del asistente y enviar deltas de texto al cliente.
3. Para entrada, crear/recuperar agente y usar followup().

16. Pipeline de herramientas

```

1 // ctx.tools expone:
2 //   - Registro de capacidades
3 //   - Transporte PTC mode
4 //   - Ruteo con pre-policy      monotonic guards      dispatch      post-
      policy      final-result
5
6 // Herramientas registradas:
7 ctx.tools.register(toolBash)    // usa ctx.shell, ctx.approval, ctx.
      jobs
8 ctx.tools.register(toolFs)      // usa ctx.fs
9 ctx.tools.register(toolWeb)     // usa ctx.web
10 ctx.tools.register(toolSubagent) // usa ctx.subagents

```

Flujo:

1. Registrar herramientas con ctx.tools.register.
2. El pipeline ejecuta: pre-policy → monotonic guards → dispatch → post-policy → final-result.
3. Eventos: tools/pre-execute → tools/execute → tools/post-execute → tools/result.

17. Flujo del turno del agente

```
1  turn/start      (pasos)      agent/turn-stopping      turn/end
```

Modos de ejecución:

1. **Inbox/Cola:** El Driver se activa con trabajo en cola. Reclama mensajes de entrada (agent/inbox/claimed).
2. **Pre-paso (Waterfall):** Emite agent/pre-step. Los hooks pueden rechazar autoritativamente o ingresar (enter(messages)) el lote propuesto. Un rechazo cierra el turno sin ejecutar paso.
3. **Paso de ejecución:** Si se ingresa, inicia step/start. Ensambla el prompt del sistema (system-prompt/assemble), realiza la solicitud al LLM (agent/request → llm/stream), y transmite los chunks de vuelta (agent/assistant-stream). El resultado exitoso se registra como assistant/message; los fallos/errores como assistant/attempt.
4. **Ejecución de herramientas:** Clasifica y ejecuta llamadas a herramientas en un pool rodante con barreras. Emite tool/call y tool/result. Si las herramientas generan más trabajo, se inicia un nuevo paso.

18. Trazabilidad append-only

- **Eventos de sesión:** turn/, step/, user/message, assistant/message, assistant/attempt, y tool/* son eventos durables e inmutables en el log de sesión.
- **Transmisión en tiempo real:** agent/assistant-stream publica chunks transitorios. El loop consolida el stream completo como un assistant/message o assistant/attempt antes de un frame de fin confirmado.
- **Recuperabilidad:** Todo lo que el modelo ve debe ser reconstruible desde el log. Si un proceso se interrumpe antes de un settlement, no queda un stream de intento persistente. Los consumidores de transcripciones deben usar session/event.

19. Almacenamiento con backends múltiples

```
1  // Backends nombrados bajo ctx.storage
2  storage.register('json', storageJson)
3  storage.register('sqlite', storageSqlite)
4
5  // ctx.storageDomain espera a cada backend configurado y publica
6  // un único servicio tipado para estado duradero
```

Flujo:

1. Registrar backends con `storage.register(nombre, implementación)`.
2. `ctx.storageDomain` enruta dominios a backends específicos.
3. Configuración: backend (default) y routes (overrides por dominio).

20. Sesiones y persistencia

```
1 // ctx.sessions: append-only, in-memory
2 // Emite feed de eventos durables
3
4 // ctx.sessionPersistence (seam)      backends reales
5 // Implementación JSONL: un artifact por Session con SessionEvent
  vocabulary
6 // Consumidores: agent-loop, tool-bash, hooks, session-query, message-
  feedback
7
8 // ctx.sessionProjectionCache: checkpoint cache + cold-read ladder
9 // Puntos mandatorios: throttled + turn/end/detach
```

Flujo:

1. `ctx.sessions` mantiene el log en memoria.
2. `ctx.sessionPersistence` persiste a JSONL.
3. `ctx.sessionProjectionCache` mantiene checkpoints para cold-read.

prerrequisitos

Prerrequisitos del proyecto dsh

Dependencias del sistema

- **Node.js:** runtime JavaScript/TypeScript.
- **pnpm:** gestor de paquetes. Debe estar en el PATH para `dsh plugin`.
- **pnpm 10:** para plugins desde Git con prepare script, bloquea hasta permitir en `pnpm-workspace.yaml`.

- **CPython 3.10+**: requerido por @deepseek-ai/dsh-experimental-code-runtime-python.

Paquetes npm principales

Core

- @deepseek-ai/cordis — kernel de plugins (Context, Service, Fiber, eventos).
- @deepseek-ai/dsh-agent — fábrica de agentes.
- @deepseek-ai/dsh-agent-loop — driver del bucle agente.
- @deepseek-ai/dsh-session — almacén de sesiones append-only.
- @deepseek-ai/dsh-tools — registro de herramientas y pipeline.
- @deepseek-ai/dsh-system-prompt — ensamblador de system prompt.

Protocolo

- @deepseek-ai/dsh-typert-protocol — RPC tipado entre Host y Cliente.
- @deepseek-ai/dsh-api-gateway — gateway de API.
- @deepseek-ai/dsh-api-remotes — remotes de API.

Bundles

- @deepseek-ai/dsh-base
- @deepseek-ai/dsh-web-app
- @deepseek-ai/dsh-headless
- @deepseek-ai/dsh-sdk-app
- @deepseek-ai/dsh-sdk-minimal
- @deepseek-ai/dsh-acp-app

Seams y backends

- @deepseek-ai/dsh-llm + dsh-llm-deepseek, dsh-llm-pi-ai, dsh-llm-replay
- @deepseek-ai/dsh-fs + dsh-fs-local, dsh-fs-sandbox, dsh-fs-e2b
- @deepseek-ai/dsh-storage + dsh-storage-json, dsh-storage-sqlite
- @deepseek-ai/dsh-subagents + dsh-subagent-spawn-in-process,
dsh-subagent-fork-in-process, dsh-subagent-acp, dsh-subagent-codex,
dsh-subagent-claude-code, dsh-subagent-dsh-sdk
- @deepseek-ai/dsh-compaction + dsh-compaction-basic
- @deepseek-ai/dsh-sandbox + dsh-sandbox-local

- @deepseek-ai/dsh-shell + dsh-bash-local, dsh-bash-sandbox, dsh-pwsh-local
- @deepseek-ai/dsh-web + dsh-web-search-exa, dsh-web-search-perplexity, dsh-web-search-deepseek, dsh-web-fetch-http

Herramientas

- @deepseek-ai/dsh-tool-bash
- @deepseek-ai/dsh-tool-fs
- @deepseek-ai/dsh-tool-web
- @deepseek-ai/dsh-tool-subagent
- @deepseek-ai/dsh-tool-terminal
- @deepseek-ai/dsh-tool-jobs
- @deepseek-ai/dsh-tool-workflow
- @deepseek-ai/dsh-tool-lsp
- @deepseek-ai/dsh-tool-skill
- @deepseek-ai/dsh-tool-cordis
- @deepseek-ai/dsh-tool-ralph
- @deepseek-ai/dsh-tool-todo

Frontend

- @deepseek-ai/dsh-host-webserver
- @deepseek-ai/dsh-client-connection
- @deepseek-ai/dsh-client-modules
- @deepseek-ai/dsh-client-hmr
- @deepseek-ai/dsh-client-ui-* (múltiples paquetes UI)

Testing

- @deepseek-ai/dsh-client-test-runtime — TestRemote, RemoteError para pruebas de cliente.
- vitest — framework de pruebas.

Utilidades

- @deepseek-ai/schemastery — validación de esquemas de configuración.
- @deepseek-ai/dsh-brand — utilidades de branding (ej. brandString).

Instalación

```
1 # Instalar dsh globalmente (o usar pnpm dlx)
2 pnpm add -g @deepseek-ai/dsh
3
4 # Verificar instalación
5 dsh --version
6
7 # Inicializar un perfil
8 dsh --profile web
9
10 # Añadir plugins a un perfil
11 dsh plugin --profile <name> add <package>
```

Requisitos de configuración

- *DSH_HOME* : directorio homedelharness (default : *~/dsh*).

Notas importantes

- Todas las configuraciones son validadas al inicio; los errores de configuración provocan fallos ruidosos.
- Los tiempos de gracia (*disposeGraceMs*, *shutdownTimeoutMs*) están acotados por *MAX_TIMER_DELAY_MS*.
- Las claves de entorno en subagentes se superponen a un entorno padre limpio de credenciales.
- El fallback SRC para desarrollo en fuente es más débil y no genera artefactos estrictos de Typert.
- Solo los paquetes que llaman a *ctx.remote.<namespace>* deben declarar la dependencia *'remote.<namespace>'* en su array *inject*.